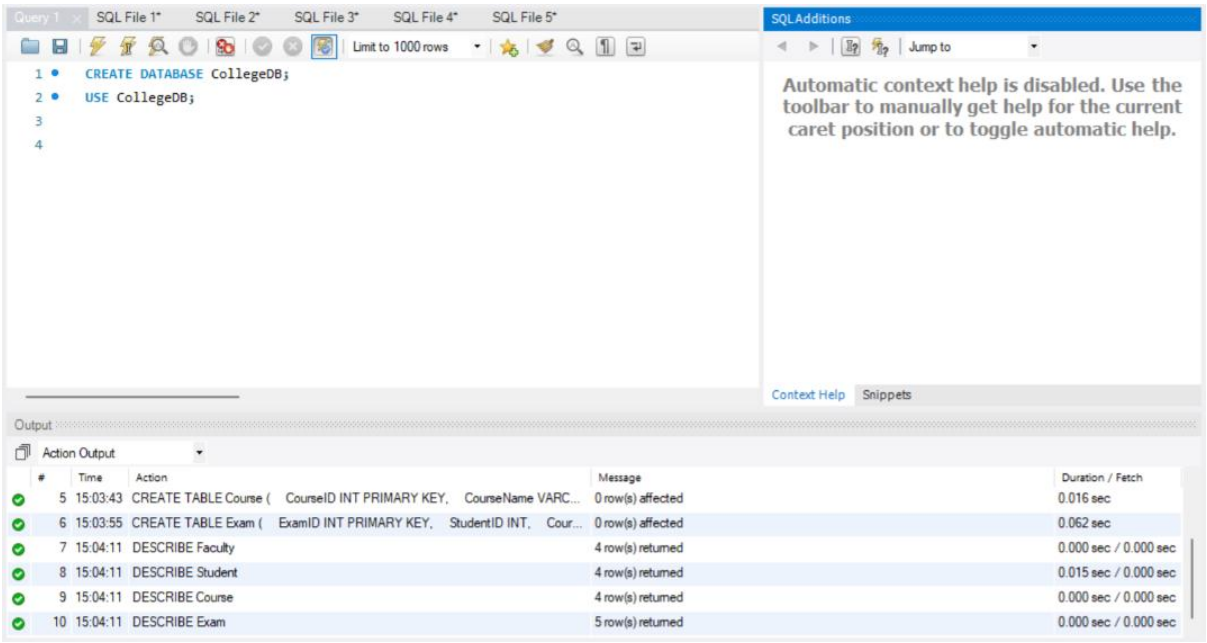
ADBMS LAB ASSESSMENT 2

QUESTION 1:

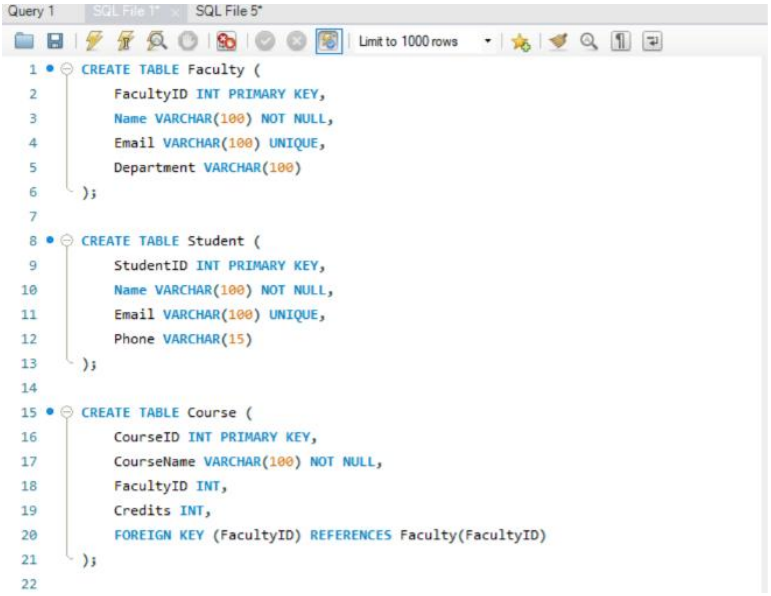
DDL Statements to Query the Table

Write DDL statements to create the four tables above. Each table must have a Primary Key. Course.FacultyID must be a Foreign Key referencing Faculty; Exam.StudentID and Exam.CourseID must be Foreign Keys referencing Student and Course respectively. Apply NOT NULL on Name fields and UNIQUE on an appropriate alternate column. Run the table creation and confirm with DESCRIBE/\d output for all four tables before proceeding.



1. Create the Database

- `CREATE DATABASE CollegeDB;` – Creates a new database named CollegeDB.
- `USE CollegeDB;` – Selects CollegeDB to perform all further SQL operations.



Create Faculty Table

- `CREATE TABLE Faculty` – Creates a table to store faculty details.
- `FacultyID INT PRIMARY KEY` – Stores a unique ID for each faculty member.
- `Name VARCHAR(100) NOT NULL` – Stores the faculty name; it cannot be empty.
- `Email VARCHAR(100) UNIQUE` – Stores unique email addresses.
- `Department VARCHAR(100)` – Stores the faculty member's department.

Create Student Table

- `CREATE TABLE Student` – Creates a table to store student details.
- `StudentID INT PRIMARY KEY` – Stores a unique ID for each student.
- `Name VARCHAR(100) NOT NULL` – Stores the student's name; it cannot be empty.
- `Email VARCHAR(100) UNIQUE` – Ensures every student has a unique email.
- `Phone VARCHAR(15)` – Stores the student's phone number.

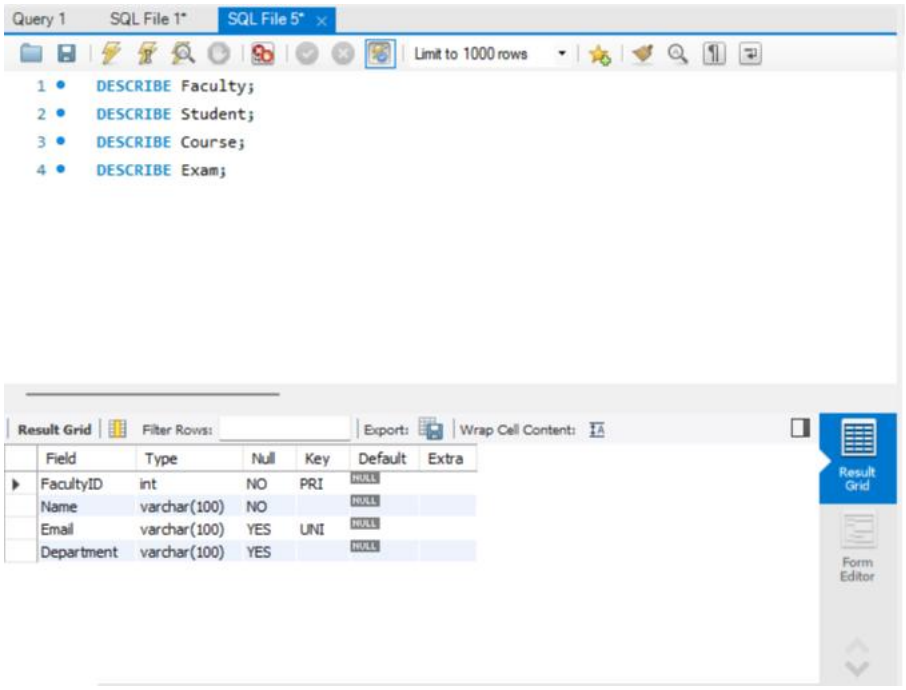
Create Course Table

- `CREATE TABLE Course` – Creates a table to store course details.
- `CourseID INT PRIMARY KEY` – Stores a unique ID for each course.
- `CourseName VARCHAR(100) NOT NULL` – Stores the course name; it cannot be empty.
- `FacultyID INT` – Stores the ID of the faculty teaching the course.
- `Credits INT` – Stores the credit value of the course.
- `FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)` – Connects each course to an existing faculty member.

Create Exam Table

- `CREATE TABLE Exam` – Creates a table to store examination records.
- `ExamID INT PRIMARY KEY` – Stores a unique ID for each exam record.
- `StudentID INT` – Stores the ID of the student appearing in the exam.

- `CourseID INT` – Stores the ID of the course for which the exam is conducted.
- `ExamDate DATE` – Stores the examination date.
- `Marks INT` – Stores the marks obtained by the student.
- `FOREIGN KEY (StudentID) REFERENCES Student(StudentID)` – Connects exam records to existing students.
- `FOREIGN KEY (CourseID) REFERENCES Course(CourseID)` – Connects exam records to existing courses.



Verify All Four Tables Using DESCRIBE

- `DESCRIBE Faculty;` – Displays the structure of the Faculty table.
- `DESCRIBE Student;` – Displays the structure of the Student table.
- `DESCRIBE Course;` – Displays the structure of the Course table.
- `DESCRIBE Exam;` – Displays the structure of the Exam table.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
StudentID	int	NO	PRI	NULL	
Name	varchar(100)	NO		NULL	
Email	varchar(100)	YES	UNI	NULL	
Phone	varchar(15)	YES		NULL	

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
CourseID	int	NO	PRI	NULL	
CourseName	varchar(100)	NO		NULL	
FacultyID	int	YES	MUL	NULL	
Credits	int	YES		NULL	

Credits

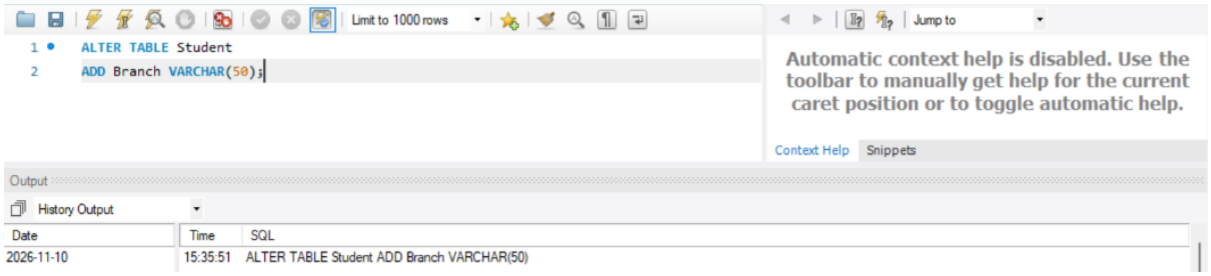
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
ExamID	int	NO	PRI	NULL	
StudentID	int	YES	MUL	NULL	
CourseID	int	YES	MUL	NULL	
ExamDate	date	YES		NULL	
Marks	int	YES		NULL	

QUESTION 2.

Inserting Data in to Tables, Update Data.

(Requires Q1 tables to exist) Insert at least 8 Students, 4 Faculty, 5 Courses and 12 Exam records, ensuring enough variety to support later GROUP BY and JOIN queries (e.g., multiple students per course, at least one student with no exam record yet). Then: (a) UPDATE one student's Branch, (b) DELETE one Exam record, (c) attempt to INSERT an Exam record with a non-existent StudentID and show/explain the resulting foreign key error.





Insert Data into Faculty Table

- `INSERT INTO Faculty` – Inserts new records into the Faculty table.
- `FacultyID, Name, Email, Department` – Specifies the columns in which data will be inserted.
- `VALUES` – Specifies the actual values to be inserted.
- `(101, 'Amit Sharma', 'amit@gmail.com', 'Computer Science')` – Inserts details of the first faculty member.
- Remaining rows – Insert details of other faculty members.

```sql
INSERT INTO Student (StudentID, Name, Email, Phone, Branch)
VALUES
(1, 'Ananya', 'ananya@gmail.com', '9876543210', 'BCA'),
(2, 'Rohit', 'rohit@gmail.com', '9876543211', 'BCA'),
(3, 'Sneha', 'sneha@gmail.com', '9876543212', 'BCA'),
(4, 'Karan', 'karan@gmail.com', '9876543213', 'BCA'),
(5, 'Mehak', 'mehak@gmail.com', '9876543214', 'BCA'),
(6, 'Arjun', 'arjun@gmail.com', '9876543215', 'BCA'),
(7, 'Ishita', 'ishita@gmail.com', '9876543216', 'BCA'),
(8, 'Vansh', 'vansh@gmail.com', '9876543217', 'BCA');
```

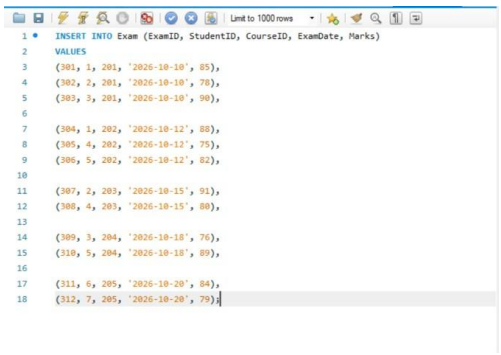
Insert Data into Student Table

- `INSERT INTO Student` – Inserts new records into the Student table.
- `StudentID, Name, Email, Phone, Branch` – Specifies the columns where student details will be stored.
- `VALUES` – Provides the actual student information.
- `(1, 'Ananya', 'ananya@gmail.com', '9876543210', 'BCA')` – Inserts details of the first student.
- Remaining rows – Insert details of other students.

```sql
INSERT INTO Course (CourseID, CourseName, FacultyID, Credits)
VALUES
(201, 'Database Management', 101, 4),
(202, 'Python Programming', 102, 3),
(203, 'Data Science', 103, 4),
(204, 'Mathematics', 104, 3),
(205, 'Computer Networks', 101, 4);
```

Insert Data into Course Table

- `INSERT INTO Course` – Inserts new records into the Course table.
- `CourseID, CourseName, FacultyID, Credits` – Specifies the columns for course information.
- `VALUES` – Provides the actual course details.
- `(201, 'Database Management', 101, 4)` – Inserts a course assigned to faculty member 101.
- Remaining rows – Insert details of other courses.



```
INSERT INTO Exam (ExamID, StudentID, CourseID, ExamDate, Marks)
VALUES
(301, 1, 201, '2026-10-10', 85),
(302, 2, 201, '2026-10-10', 78),
(303, 3, 201, '2026-10-10', 90),

(304, 1, 202, '2026-10-12', 88),
(305, 4, 202, '2026-10-12', 75),
(306, 5, 202, '2026-10-12', 82),

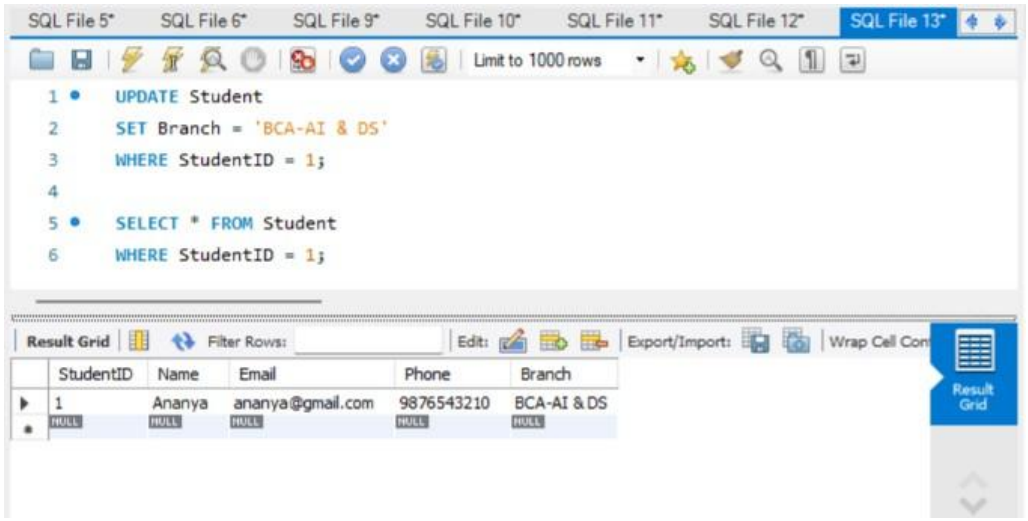
(307, 2, 203, '2026-10-15', 91),
(308, 4, 203, '2026-10-15', 80),

(309, 3, 204, '2026-10-18', 76),
(310, 5, 204, '2026-10-18', 89),

(311, 6, 205, '2026-10-20', 84),
(312, 7, 205, '2026-10-20', 79);
```

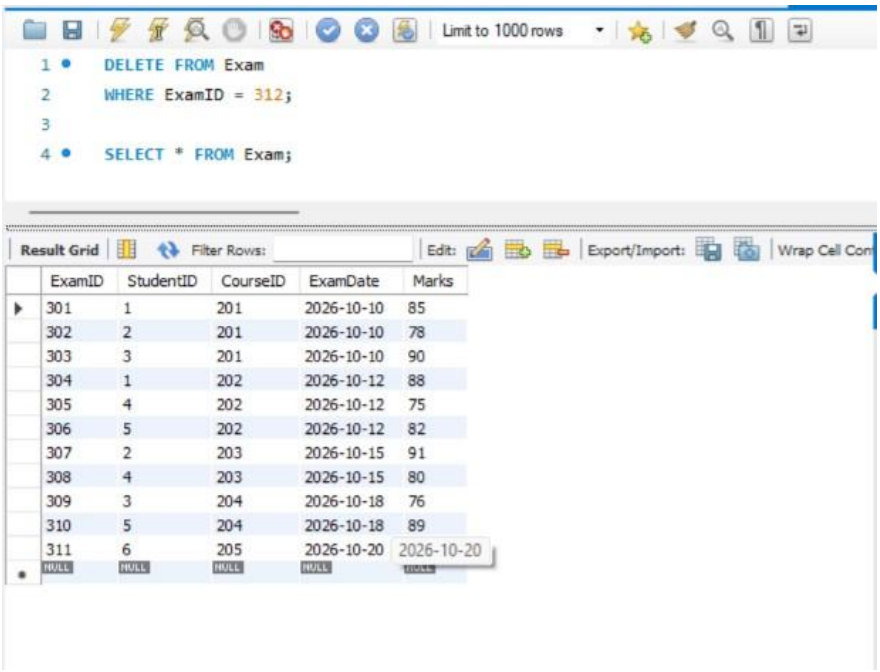
Insert Data into Exam Table

- `INSERT INTO Exam` – Inserts examination records into the Exam table.
- `ExamID, StudentID, CourseID, ExamDate, Marks` – Specifies the columns for examination details.
- `VALUES` – Provides the actual examination records.
- `(301, 1, 201, '2026-10-10', 85)` – Inserts an exam record for Student 1 in Course 201 with 85 marks.
- Remaining rows – Insert other examination records.



UPDATE Student Branch

- `UPDATE Student` – Modifies existing records in the Student table.
- `SET Branch = 'BCA-AI & DS'` – Changes the student's branch to BCA-AI & DS.
- `WHERE StudentID = 1` – Specifies the student whose branch needs to be updated.
- `SELECT *` – Displays all columns of the selected record.
- `WHERE StudentID = 1` – Displays only the record of Student 1.

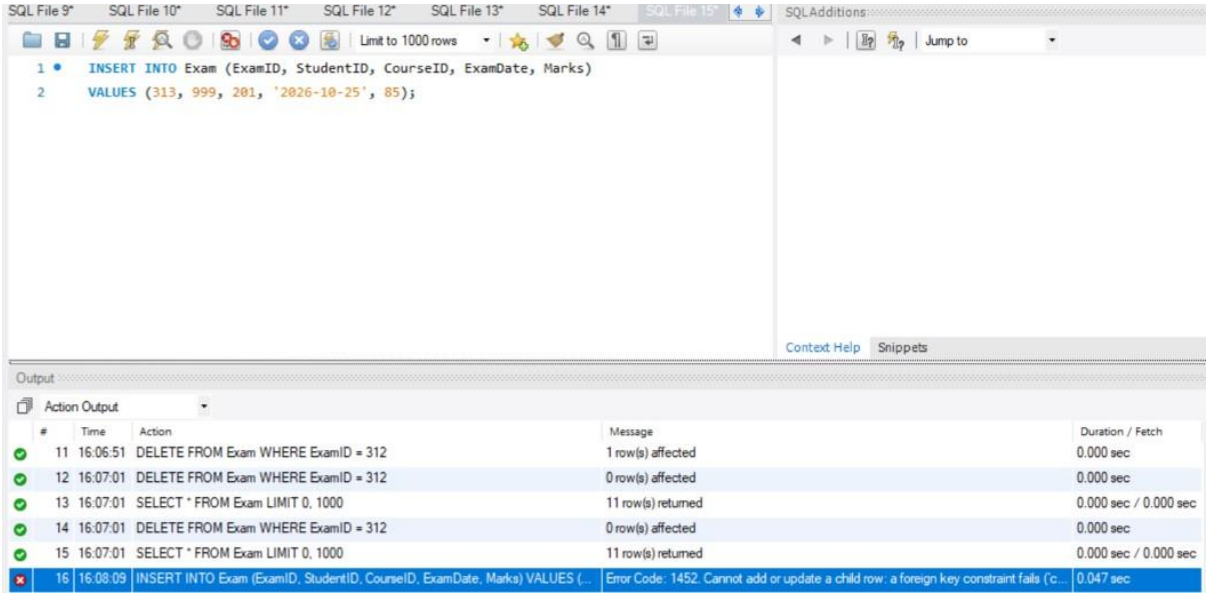


DELETE an Exam Record

- `DELETE FROM Exam` – Deletes a record from the Exam table.
- `WHERE ExamID = 312` – Specifies the exam record to be deleted.

`-SELECT * FROM Exam;`

- `SELECT * FROM Exam` – Displays all remaining records in the Exam table.



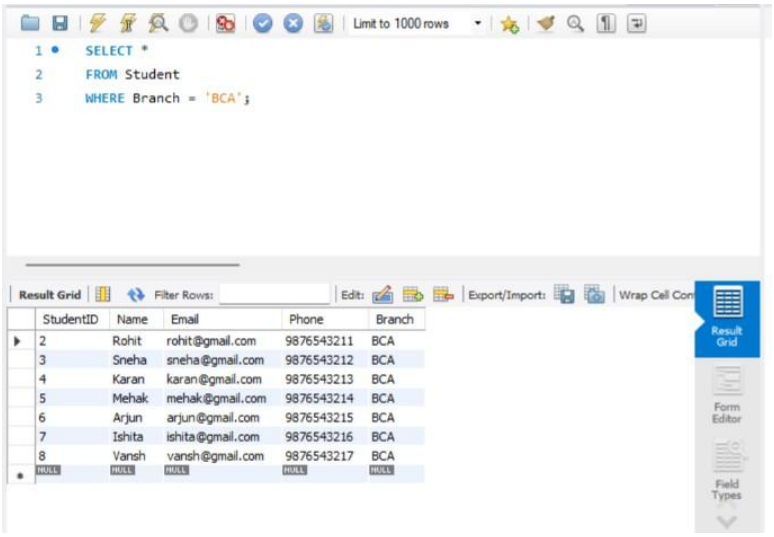
Test Foreign Key Constraint

- `INSERT INTO Exam` – Attempts to insert a new examination record.
- `StudentID = 999` – Uses a student ID that does not exist in the Student table.
- `CourseID = 201` – Specifies an existing course.
- `VALUES` – Provides the examination details to be inserted.
- `INSERT statement` – Fails because the referenced StudentID does not exist.

QUESTION 3.

Implementation of Where Clause

(Uses Q1–Q2 data only — no new tables/data) Write and run: (a) all students from a specific Branch using WHERE, (b) all exam records sorted by Marks descending using ORDER BY, (c) average Marks per CourseID using GROUP BY and AVG, (d) the highest Marks scored in each Course using GROUP BY and MAX.



- `SELECT *` – Displays all columns from the table.
- `FROM Exam` – Specifies the Exam table.
- `ORDER BY Marks` – Sorts the examination records according to marks.
- `DESC` – Arranges the marks from highest to lowest.

```
1 •  SELECT *
2    FROM Exam
3    ORDER BY Marks DESC;
```

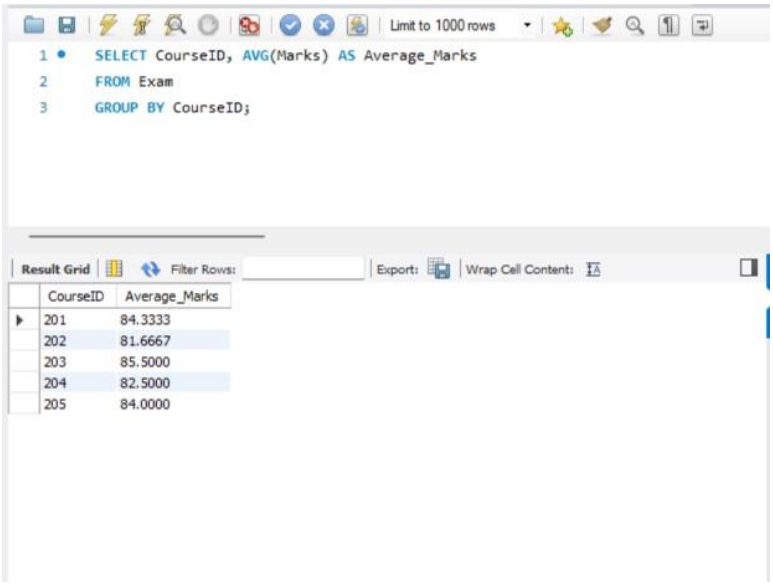
Limit to 1000 rows

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Con

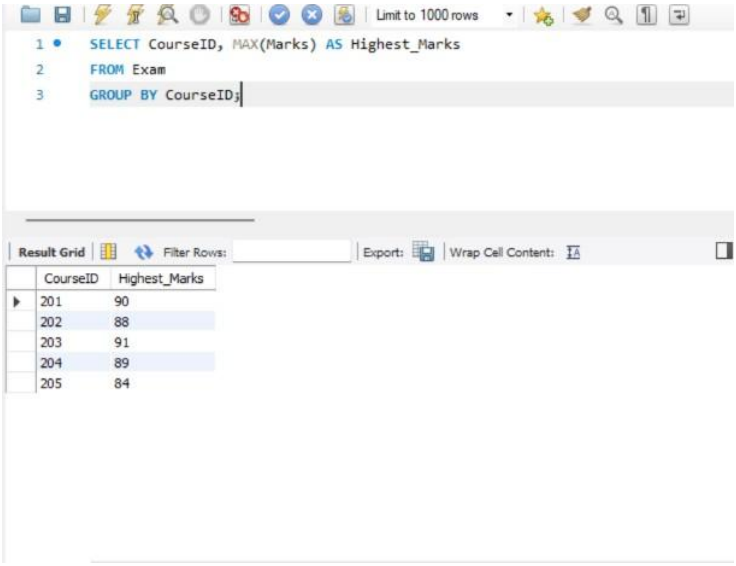
ExamID	StudentID	CourseID	ExamDate	Marks
307	2	203	2026-10-15	91
303	3	201	2026-10-10	90
310	5	204	2026-10-18	89
304	1	202	2026-10-12	88
301	1	201	2026-10-10	85
311	6	205	2026-10-20	84
306	5	202	2026-10-12	82
308	4	203	2026-10-15	80
302	2	201	2026-10-10	78
309	3	204	2026-10-18	76
305	4	202	2026-10-12	75
NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor | Field Types

- `SELECT *` – Displays all columns from the table.
- `FROM Exam` – Specifies the Exam table.
- `ORDER BY Marks` – Sorts the examination records according to marks.
- `DESC` – Arranges the marks from highest to lowest.



- `SELECT CourseID` – Displays the course ID.
- `AVG(Marks)` – Calculates the average marks obtained in each course.
- `AS Average_Marks` – Gives a name to the calculated average column.
- `FROM Exam` – Retrieves records from the Exam table.
- `GROUP BY CourseID` – Groups exam records according to their course ID.



- `SELECT CourseID` – Displays the course ID.
- `MAX(Marks)` – Finds the highest marks obtained in each course.
- `AS Highest_Marks` – Gives a name to the highest marks column.
- `FROM Exam` – Retrieves records from the Exam table.
- `GROUP BY CourseID` – Groups exam records according to their course ID.